CA 1 - Practical ML: Study Notes

Short, practical notes for the 1.5-hour practical CA. Code templates are in the **code/** folder (01_eda.py ... 06_text_classification.py) - open them in VS Code / Jupyter and run cell by cell.

What you must justify (from the CA brief)

Section	What the examiner wants to see
1. Type of method	Supervised (classification / regression) or unsupervised (clustering), and WHY.
2. EDA	Shape, types, missing values, target balance, distributions, correlations + your comments.
3. Preprocessing / features	What you cleaned / encoded / scaled / dropped, and why.
4. ML pipeline	The steps in order: load - clean - split - scale - train - evaluate - compare.
5. Results	Metrics table for 2-3 models, best model, what the numbers mean.
6. Ethical concerns	Bias, privacy, transparency, misuse - linked to THIS dataset.

Time plan (90 min)

Minutes	Do this
0-10	Read data, identify the target, decide method type. Write section 1.
10-30	EDA (01_eda.py). Write 3-5 bullet findings under the plots.
30-45	Preprocessing + feature selection (02_preprocessing.py).
45-70	Train 2-3 models, evaluate, compare (03 / 04 / 05 / 06).
70-85	Results table + ethical concerns.
85-90	Re-run all cells top to bottom (Restart & Run All). Save.

Tip: Marks come from **justification**. After every step write one sentence: *what I did* and *why*.

1. Choosing the method

Step 1 - look at the target column

What you see	Problem type	Methods
Target is a category (yes/no, spam/ham, 3 species)	Classification (supervised)	Logistic Regression, Decision Tree, Random Forest, KNN, Naive Bayes
Target is a number (price, salary)	Regression (supervised)	Linear Regression, Decision Tree / Random Forest Regressor
No target - 'find groups / segments'	Clustering (unsupervised)	KMeans
Target + a text column (reviews, SMS)	Text classification	Bag of Words / TF-IDF + Naive Bayes or Logistic Regression

Step 2 - which model and when

Model	Use when	Watch out
Linear Regression	Predict a number; relationship looks roughly linear; need to explain coefficients.	Sensitive to outliers; can't capture curves.
Logistic Regression	Binary/multi-class; good first model; easy to explain.	Needs scaling; linear boundary.
Decision Tree	Need simple rules you can show; mixed data types.	Overfits - limit max_depth.
Random Forest	Usually best accuracy on tables; gives feature importance.	Less interpretable, slower.
KNN	Small data, simple patterns.	MUST scale; slow on big data.
Naive Bayes	Text data (word counts).	Assumes features independent.
KMeans	Group similar rows; no labels.	Must scale; choose k (elbow/silhouette).

Example justification: *"The target 'approved' has two values (0/1), so this is a binary classification problem. I use Logistic Regression as an interpretable baseline and Random Forest because it handles non-linear relationships and gives feature importance."*

2. EDA template

Goal: understand the data **before** modelling. Always write what you notice. Full code: **code/01_eda.py**

```
df = pd.read_csv("data.csv")
df.shape                      # rows, columns
df.head()                     # first rows
df.info()                     # types + non-null counts
df.describe()                 # mean, min, max ... (numeric)
df.isnull().sum()             # missing values per column
df.duplicated().sum()         # duplicate rows
df["target"].value_counts()   # class balance
df.hist(figsize=(10, 8))                                  # distributions
sns.boxplot(data=df[["age", "income"]])                   # outliers
sns.heatmap(df.corr(numeric_only=True), annot=True)       # correlations
sns.countplot(x="category_col", hue="target", data=df)    # category vs target
```

Check	What it tells you	Example finding to write
shape / info	Size, column types	"2000 rows, 8 columns: 5 numeric, 3 categorical."
isnull	Missing data	"income has 5% missing - will fill with median."
value_counts(target)	Class balance	"54% approved vs 46% - fairly balanced, accuracy is OK."
hist / boxplot	Skew and outliers	"income is right-skewed with high outliers."
heatmap	Strong predictors, duplicates of info	"credit_score has the strongest correlation with the target."

3. Preprocessing and feature selection

Full code: **code/02_preprocessing.py**

Problem	Fix	Code
Duplicate rows	Remove	df = df.drop_duplicates()
ID / name / address columns	Drop (no pattern, privacy risk)	df = df.drop(columns=['id'])
Missing numeric	Fill with median	df['a'] = df['a'].fillna(df['a'].median())
Missing category	Fill with mode	df['b'] = df['b'].fillna(df['b'].mode()[0])
Too much missing (>50%)	Drop the column	df = df.drop(columns=['c'])
Text categories	One-hot encode	pd.get_dummies(df, columns=['b'], drop_first=True)
Different scales	StandardScaler (LogReg, KNN, KMeans)	scaler.fit_transform(X_train)
Imbalanced classes	stratify + look at recall/F1	train_test_split(..., stratify=y)

Correct order (avoid data leakage)

```
X = df.drop(columns=["target"])
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)   # fit on TRAIN only
X_test  = scaler.transform(X_test)        # only transform test
```

Tip: Data leakage = test data influencing training (e.g. scaling before splitting). It makes results look better than they really are. Always split first. Trees / Random Forest do not need scaling.

Feature selection - 3 simple ways

Method	Code	When
Correlation	df.corr(numeric_only=True)['target']	Quick check in EDA
SelectKBest	SelectKBest(f_classif, k=5).fit(X_train, y_train)	Keep top-k features (f_regression for numbers)
Feature importance	RandomForestClassifier().fit(...).feature_importances_	See which features the model uses

Also drop: features with no variation, and one of two features that are almost the same (correlation > 0.9).

4. Train and evaluate models

Every sklearn model works the same way: **create - fit - predict - score**.

```
model = LogisticRegression()
model.fit(X_train, y_train)              # learn from training data
pred = model.predict(X_test)             # predict unseen data
print(accuracy_score(y_test, pred))      # compare with real answers

scores = cross_val_score(LogisticRegression(), X_train, y_train, cv=5)   # 5-fold CV
print(scores.mean())                     # more reliable than one split
```

Which metric?

Task	Metric	Meaning / when to use
Classification	Accuracy	% correct. Only when classes are balanced.
	Precision	Of predicted positives, how many are right. Use when false alarms are costly (spam filter).
	Recall	Of real positives, how many found. Use when missing one is costly (disease, fraud).
	F1-score	Balance of precision and recall. Best single number for imbalanced data.
	Confusion matrix	Shows TP, TN, FP, FN - always include it.
Regression	MAE	Average error in the target's units (easy to explain).
	RMSE	Like MAE but punishes big errors more.
	R2	Share of variance explained. 1 = perfect, 0 = no better than the mean.
Clustering	Elbow / Silhouette	Pick k where the elbow bends / silhouette is highest (max 1).

Overfitting check

Compare train and test score. Train 0.99 vs test 0.75 = **overfitting** - simplify the model (e.g. DecisionTree(max_depth=5)) or use Random Forest.

```
print("train:", model.score(X_train, y_train), "test:", model.score(X_test, y_test))
```

Writing the results

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.72	0.73	0.75	0.74
Decision Tree	0.69	...	...	...
Random Forest	0.71	...	...	...

Example: *"Logistic Regression performed best (F1 = 0.74). credit_score and income were the most important features. The model makes more false positives than false negatives, which in loan approval means some risky applicants get approved."*

5. Ethical concerns (Human-Centred AI)

Always link the concern to **your** dataset. Name the concern, give an example, and suggest a fix.

Concern	Example	Mitigation
Bias / fairness	Loan model approves fewer women because past data was biased.	Remove sensitive columns, check results per group, balance the data.
Proxy variables	Postcode or income can stand in for race/gender even after removing them.	Check per-group results, not just drop columns.
Privacy	Names, addresses, health data in the dataset.	Drop / anonymise personal data; GDPR consent.
Transparency	Applicant rejected by a 'black box' with no reason.	Use interpretable models or show feature importance.
Unbalanced data	Fraud is 1% of rows - 99% accuracy by predicting 'no fraud'.	Use recall / F1, stratify, class_weight='balanced'.
Accountability	Who is responsible for a wrong medical prediction?	Human-in-the-loop: model supports, human decides.
Misuse / generalisation	Model trained on one country used in another.	State limits of the data; monitor after deployment.

Simple fairness check (from 03_classification.py)

```
results = pd.DataFrame({"actual": y_test, "predicted": pred, "gender": gender.loc[y_test.index]})
for g in ["F", "M"]:
    group = results[results["gender"] == g]
    print(g, accuracy_score(group["actual"], group["predicted"]), group["predicted"].mean())
# similar accuracy and approval rate for each group = fairer model
```

Example paragraph: *"The dataset contains gender, a sensitive attribute. I removed it from the features but kept it to audit the model. Approval rates were 56% for F and 53% for M, so the gap is small, but income may act as a proxy. Because this model affects people's access to credit, it should be explainable and a human should review rejections."*

6. Practice tasks

Do these with the code templates. Datasets are already in your repo. Time yourself - aim for the CA pace.

Task 1 - EDA (15 min)

Data: loan_practice.csv (run code/make_practice_data.py first)

[ ] Find shape, types, missing values and duplicates.

[ ] Plot target balance and the correlation heatmap.

[ ] Write 5 bullet findings in your own words.

Task 2 - Preprocessing (15 min)

Data: loan_practice.csv

[ ] Fill missing values (median / mode) and one-hot encode categories.

[ ] Split 80/20 with stratify, then scale correctly.

[ ] Use SelectKBest(k=5) and RF importance. Do they agree?

Task 3 - Classification (20 min)

Data: loan_practice.csv

[ ] Train Logistic Regression, Decision Tree and Random Forest.

[ ] Make a results table (accuracy, precision, recall, F1).

[ ] Check overfitting: compare train vs test score of the Decision Tree with and without max_depth.

Task 4 - Ethics (10 min)

Data: loan_practice.csv

[ ] Run the fairness check per gender. Write the example paragraph for your result.

[ ] Name one proxy variable and explain why it is a risk.

Task 5 - Regression (20 min)

Data: experiments/linear_regression/Housing.csv (and Salary_data.csv)

[ ] Drop Address and say why (text + privacy).

[ ] Linear Regression vs Random Forest: report MAE, RMSE, R2.

[ ] Which feature affects price the most?

Task 6 - Clustering (15 min)

Data: Iris (built in) - ignore the target

[ ] Elbow + silhouette: choose k and justify it.

[ ] Describe each cluster in one sentence using groupby().mean().

Task 7 - Text (20 min)

Data: experiments/spam_classifier/SMSSpamCollection.txt or movie_reviews(in).csv

[ ] Compare BoW + Naive Bayes vs TF-IDF + Logistic Regression.

[ ] Data is imbalanced (87% ham) - which metric do you report and why?

[ ] Ethics: what is the cost of a false positive (real message sent to spam)?

Task 8 - Full mock CA (90 min)

Data: Pick a dataset you have not used (e.g. sklearn load_breast_cancer / load_wine)

[ ] Answer all 6 CA sections in one notebook, with a sentence of justification after every step.

[ ] Finish with Restart & Run All.

Self-check questions

- Why split before scaling? (data leakage)
- Why is accuracy misleading on imbalanced data? (predicting the majority class scores high)
- When do you prefer recall over precision? (missing a positive is costly: disease, fraud)
- Why does KMeans need scaling? (it uses distance; big-unit columns dominate)
- Why can removing 'gender' still leave bias? (proxy variables)
- What does R2 = 0.9 mean? (model explains 90% of variance in the target)